

FOOD DELIVERY DATABASE PROJECT

By Rotea Dragoş (Group 30424)

1. Description of the Topic and Purpose

My project consists of a relational database designed for a food delivery application, with the purpose of managing the data for an app created for the Java Oriented-Object Programming assignment. The system is designed to be highly flexible and is currently modeled to handle food deliveries. Nevertheless, the underlying architecture can be easily adapted for any other online marketplace (electronics, pharmacy, clothing etc.).

Since I study in Cluj-Napoca, I have populated the database so as to reflect the local market as closely as possible, using real restaurant names like 'Marty' or 'Zama' and real-life street addresses to ensure the data is coherent and consistent with the reality of the city.

2. Identification of Entities and Justification

My database consists of **5 tables** which effectively shape my project into a well interlinked data storage for the app, with meaningful relationships between each piece of data:

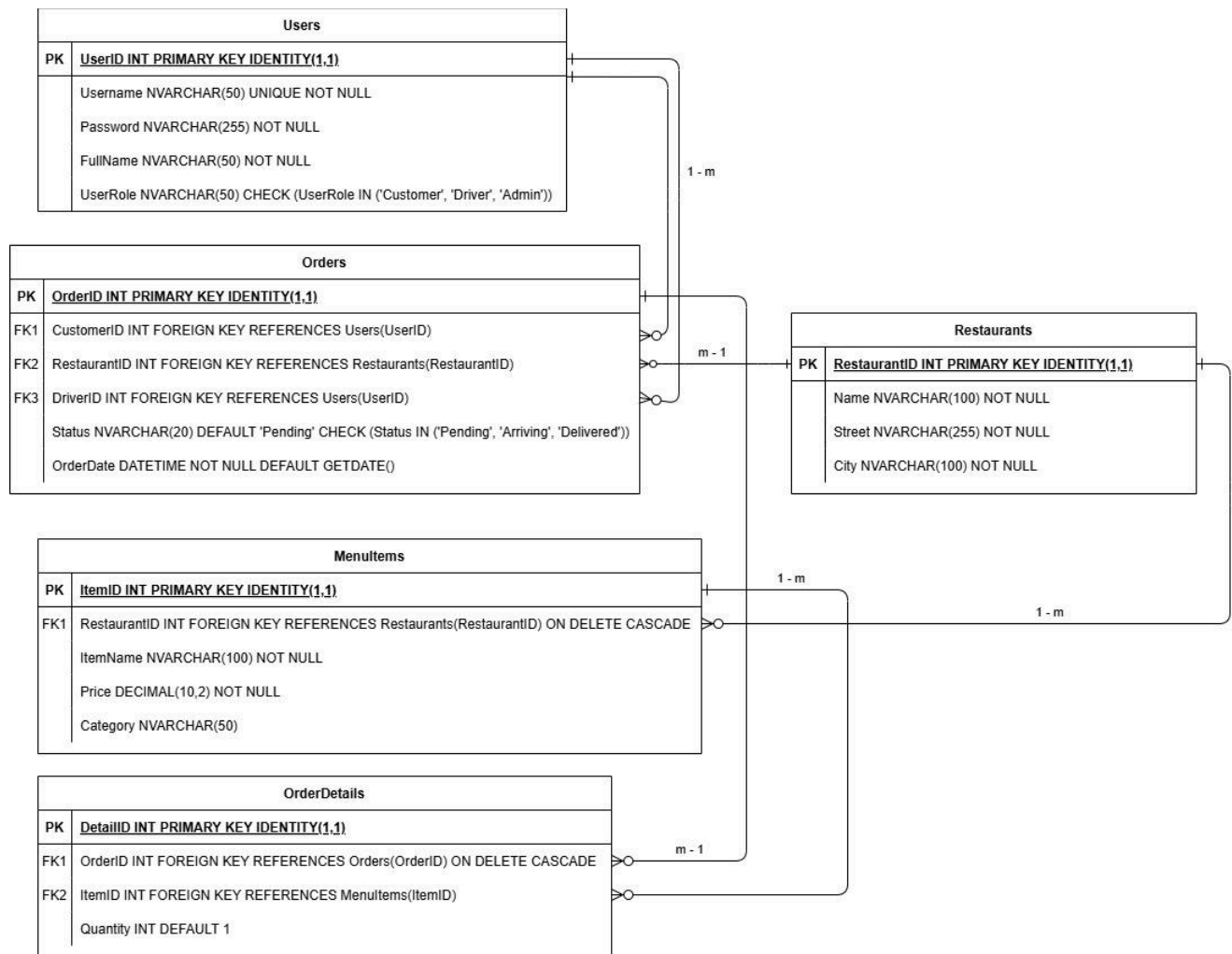
- **Users:** Acts as a container for everyone accessing the app. By including a UserRole attribute, we handle three distinct personas who can use our app (Customers, Drivers and Admins) within a single structure.
- **Restaurants:** Stores the primary data for the vendors and food shops, including unique IDs and precise locations in Cluj-Napoca and its proximity.
- **MenuItems:** This table bridges restaurants to their products and menu. It allows us to categorize the items with different labels (Soup, Main, Dessert, Japanese etc.) and manage the prices.

- **Orders:** Represents the central transaction hub of the database, interlinking Users (Customers and Drivers) with Restaurants. It also tracks the lifecycle of a delivery through the updatable Status field.
- **OrderDetails:** A specialized bridge table used to handle the relationship between Orders and MenuItems, allowing a single order to contain multiple different dishes. It contains the items added to a specific order, from a specific restaurant.

3. Conceptual model (ERD)

In terms of relationships in my database, this is how each of them is modeled, followed by the ERD (entity relationship diagram):

- **Users to Orders (1:N):** One Customer can place multiple orders over time (One-To-Many Relationship). However, each specific order belongs to only one customer. We also have the secondary 1:N relationship between Drivers and Orders, where one driver can deliver many orders, but a specific order can only be delivered by only one driver.
- **Restaurants to MenuItems (1:N):** Each restaurant manages its own menu. One restaurant can have a list of menu items, but a specific item only belongs to the respective restaurant entry.
- **Restaurants to Orders (1:N):** A restaurant can receive hundreds of orders, but each order is placed at one specific restaurant at a time.
- **Orders to OrderDetails (1:N):** This composition relationship means that each order has multiple entries of order details (like lines in a receipt), while one specific entry can only belong to that respective order.
- **MenuItems to OrderDetails (1:N):** A specific dish can be part of many different orders placed by different people.



4. Logic model (technical specifications)

a) **Users** Table:

- **UserID** (INT): Primary Key, IDENTITY(1,1) → we automatically increment to ensure unique identification for each person.
- **Username** (NVARCHAR(50)): UNIQUE, NOT NULL → prevents two people from having the same login
- **Password** (NVARCHAR(255)): NOT NULL → capacity is 255 to allow future password hashing for a real-life project

- **FullName** (NVARCHAR(100)): NOT NULL → real name of the user
- **UserRole** (NVARCHAR(20)): NOT NULL → includes check constraint ('Customer', 'Driver', 'Admin') to prevent invalid roles from entering the system

b) **Restaurants** Table:

- **RestaurantID** (INT): Primary Key, IDENTITY(1,1)
- **Name** (NVARCHAR(100)): NOT NULL
- **Street** (NVARCHAR(255)): NOT NULL → mandatory to ensure drivers know where to pick-up the food
- **City** (NVARCHAR(100)): NOT NULL → Cluj-Napoca proximity

c) **MenuItems** Table:

- **ItemID** (INT): Primary Key, IDENTITY(1,1)
- **RestaurantID** (INT): Foreign Key (References Restaurant) → includes ON DELETE CASCADE → if a restaurant is deleted, its menu is automatically deleted as well to prevent "orphan" items
- **ItemName** (NVARCHAR(100)): NOT NULL
- **Price** (DECIMAL(10,2)): NOT NULL → precision choice: DECIMAL instead of FLOAT to ensure exact calculations for transactions and avoid rounding errors
- **Category** (NVARCHAR(50)): NOT NULL → used for filtering (ex. Pizza, Desert, Coffee)

d) **Orders** Table:

- **OrderID** (INT): Primary Key, IDENTITY(1,1)
- **CustomerID** (INT): Foreign Key (References Users), NOT NULL → an order must have a buyer

- **RestaurantID** (INT): Foreign Key (References Restaurant), NOT NULL → an order must have a source
- **DriverID** (INT): Foreign Key (References Users), Nullable → it remains NULL until a driver accepts the order
- **Status** (NVARCHAR(20)): DEFAULT 'Pending' → constraint check ('Pending', 'Arriving', 'Delivered')
- **OrderDate** (DATETIME): NOT NULL → defaults to the current system time to track when the transaction occurred

e) **OrderDetails** Table:

- **DetailID** (INT): Primary Key, IDENTITY(1,1)
- **OrderID** (INT): Foreign Key (References Orders) → includes ON DELETE CASCADE
- **ItemID** (INT): Foreign Key (References MenuItems): NOT NULL
- **Quantity** (INT): NOT NULL, DEFAULT 1

I have built the logic of the database on **3 architectural pillars**:

1. **Referential Integrity** → we ensure that relationships between tables remain consistent. Every foreign key must point to an existing, valid primary key. We prevent "orphan items" by implementing ON DELETE CASCADE for OrderDetails and MenuItems, ensuring that if a restaurant or an order is removed, the system automatically deletes all the related sub-data.
2. **Domain Integrity** → we ensure that data stored in a specific column is "sensible" and falls within certain rules. We apply this through data types and check constraints like for UserRole and Status.
3. **Data Consistency & Precision** → we ensure accuracy of the data over time by choosing DECIMAL(10,2) data type for Price in MenuItems. Unlike FLOAT or REAL data types we don't approximate, and choose a fixed-point type.

5. Normalization up to 3NF

In this section, I will explain how I transformed the data into a structured format to eliminate anomalies and errors:

a. First Normal Form (1NF):

REQUIREMENT: Each cell must contain a single, atomic value, and there must be a unique identifier (Primary Key).

SITUATION: Somebody might want to store the data in the Items category like "Pizza, Cola, Fries" and this violates the 1NF. The same situation could happen in a column called Menu in the Restaurant table, containing "Pizza, Pasta, Cola".

SOLUTION: I created the OrderDetails table and every row contains only one item per order with no "," being used between items in a cell. I also ensured that each table has an INT PRIMARY KEY IDENTITY(1,1). Also, instead of creating a Menu column in Restaurant, we create a new table called MenuItem, which stores in each row a new type of food or drink in the menu.

b. Second Normal Form (2NF):

REQUIREMENT: Builds upon 1NF and all non-key attributes must depend on the entire primary key. (removing partial dependencies)

SITUATION: This applies specifically to the OrderDetails table. If we stored the restaurant name inside OrderDetails, it would be a violation because the restaurant name depends on the RestaurantID, not the DetailID.

SOLUTION: Non-key attributes like Quantity depend strictly on the DetailID. All descriptive data (like names and prices) was moved to their own parent tables.

c. Third Normal Form (3NF):

REQUIREMENT: Builds upon 2NF and has no transitive dependencies (a non-key attribute depending on another non-key attribute).

SITUATION: Imagine if the Orders table had a column for DriverPhoneNumber. This is a violation because the phone number depends on DriverID, which is not the Primary Key of the Orders table. If the driver changes their phone number, we would have to update tens of rows in the Orders table.

SOLUTION: We remove all driver and customer details from the Orders table, leaving only their IDs. To see a name or phone number, we perform a JOIN with the Users table. This ensures information is stored in exactly one place, and needs to be updated only there.

6. SQL scripts:

a. DataBase CREATION:

```
CREATE DATABASE FoodDeliveryDB;
```

b. Tables CREATION:

The N prefix is used during data insertion to explicitly define string constants as Unicode, ensuring that Romanian diacritics (such as ș, ț, or ă) are preserved without corruption and avoiding the extra work of implicit conversion when saving to NVARCHAR columns.

1. Users Table:

```
CREATE TABLE Users (  
    UserID INT PRIMARY KEY IDENTITY(1,1),  
    Username NVARCHAR(50) UNIQUE NOT NULL,  
    Password NVARCHAR(255) NOT NULL,  
    FullName NVARCHAR(100) NOT NULL,  
    UserRole NVARCHAR(20) NOT NULL CHECK (UserRole IN ('Customer', 'Driver',  
'Admin'))  
);
```

2. Restaurants Table:

```
CREATE TABLE Restaurants (  
    RestaurantID INT PRIMARY KEY IDENTITY(1,1),
```

```
Name NVARCHAR(100) NOT NULL,  
Street NVARCHAR(255) NOT NULL,  
City NVARCHAR(100) NOT NULL  
);
```

3. MenuItems Table:

```
CREATE TABLE MenuItems (  
    ItemID INT PRIMARY KEY IDENTITY(1,1),  
    RestaurantID INT NOT NULL FOREIGN KEY REFERENCES  
Restaurants(RestaurantID) ON DELETE CASCADE,  
    ItemName NVARCHAR(100) NOT NULL,  
    Price DECIMAL(10, 2) NOT NULL,  
    Category NVARCHAR(50) NOT NULL  
);
```

4. Orders Table:

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY IDENTITY(1,1),  
    CustomerID INT NOT NULL FOREIGN KEY REFERENCES Users(UserID),  
    RestaurantID INT NOT NULL FOREIGN KEY REFERENCES  
Restaurants(RestaurantID),  
    DriverID INT FOREIGN KEY REFERENCES Users(UserID) NULL,  
    Status NVARCHAR(20) NOT NULL DEFAULT 'Pending' CHECK (Status IN  
('Pending', 'Arriving', 'Delivered'))  
    OrderDate DATETIME NOT NULL DEFAULT GETDATE()  
);
```

5. OrderDetails Table:

```
CREATE TABLE OrderDetails (  
    DetailID INT PRIMARY KEY IDENTITY(1,1),  
    OrderID INT NOT NULL FOREIGN KEY REFERENCES Orders(OrderID) ON  
DELETE CASCADE,  
    ItemID INT NOT NULL FOREIGN KEY REFERENCES MenuItems(ItemID),  
    Quantity INT NOT NULL DEFAULT 1 CHECK (Quantity > 0)  
);
```


c. Data INSERT:

1. Insert User Data: (25 entries)

```
INSERT INTO Users (FullName, Username, Password, UserRole) VALUES
(N'Andrei Ionescu', 'andrei_admin', 'adminPass!55', 'Admin'),
(N'Maria Georgescu', 'maria_g', 'cluj_secure7', 'Customer'),
(N'Alexandru Matei', 'alex_m', 'driver_pass1', 'Driver'),
(N'Elena Dumitru', 'elena_d', 'pass1234', 'Customer'),
(N'Cristian Stoica', 'cris_s', 'speedy_driver', 'Driver'),
(N'Ioana Radu', 'ioana_r', 'ioana_cluj', 'Customer'),
(N'Stefan Popa', 'stefan_p', 'stefan_88', 'Customer'),
(N'Denisa Marin', 'deni_m', 'delivery_44', 'Driver'),
(N'Robert Stan', 'robert_s', 'rob_stan_pass', 'Customer'),
(N'Laura Dinu', 'laura_d', 'laur_dinu1', 'Customer'),
(N'Marius Vlad', 'marius_v', 'vlad_marius_99', 'Driver'),
(N'Ana Baci', 'ana_b', 'ana_baci_pw', 'Customer'),
(N'George Luca', 'george_l', 'george_pass', 'Customer'),
(N'Alina Nistor', 'alina_n', 'fast_alina', 'Driver'),
(N'Victor Sandu', 'victor_s', 'victor_cluj', 'Customer'),
(N'Diana Voicu', 'diana_v', 'diana_v_2026', 'Customer'),
(N'Paul Diaconu', 'paul_d', 'paul_drive', 'Driver'),
(N'Silvia Albu', 'silvia_a', 'silvia_albu_7', 'Customer'),
(N'Gabriel Mocanu', 'gabriel_m', 'gabi_pass', 'Customer'),
(N'Oana Lungu', 'oana_l', 'oana_delivery', 'Driver'),
(N'Florin Rusu', 'florin_r', 'florin_rusu_1', 'Customer'),
(N'Mihaela Ene', 'miha_e', 'mihaela_ene', 'Customer'),
(N'Dan Chiriac', 'dan_c', 'dan_chiriac_9', 'Driver'),
(N'Simona Lazăr', 'simona_l', 'simona_l_pass', 'Customer'),
(N'Bogdan Ursu', 'bogdan_admin', 'admin_ursu_!1', 'Admin');
```

2. Insert Restaurants Data: (25 entries)

```
INSERT INTO Restaurants (Name, Street, City) VALUES
(N'Zama', N'Strada Napoca 16', N'Cluj-Napoca'),
(N'Marty Restaurants', N'Strada Victor Babeş 39', N'Cluj-Napoca'),
(N'Samsara Foodhouse', N'Strada Cardinal Iuliu Hossu 3', N'Cluj-Napoca'),
```

(N'Pizzeria Euphoria', N'Calea Turzii 7', N'Cluj-Napoca'),
 (N'Cimbru Author Eatery', N'Strada Decebal 37', N'Cluj-Napoca'),
 (N'Bujole', N'Piața Unirii 15', N'Cluj-Napoca'),
 (N'Indigo Indian', N'Strada Piezișă 10', N'Cluj-Napoca'),
 (N'MEAT UP', N'Strada Eroilor 14', N'Cluj-Napoca'),
 (N'Garlic Bites', N'Strada Inocențiu Micu Klein 11', N'Cluj-Napoca'),
 (N'Tokyo Japanese', N'Strada Gheorghe Marinescu 5', N'Cluj-Napoca'),
 (N'Express Chef', N'Strada Avram Iancu 492', N'Florești'),
 (N'Pizza Mania', N'Strada Eroilor 15', N'Florești'),
 (N'Burger Island', N'Strada Cetății 2', N'Florești'),
 (N'Traditional Apahida', N'Strada Libertății 20', N'Apahida'),
 (N'Wok'n'Roll', N'Iulius Mall Food Court', N'Cluj-Napoca'),
 (N'Noodle Pack', N'Vivo! Mall', N'Cluj-Napoca'),
 (N'Eggcetera', N'Strada Napoca 13', N'Cluj-Napoca'),
 (N'Roots', N'Bulevardul Eroilor 4', N'Cluj-Napoca'),
 (N'Olivo Coffee Roasters', N'Bulevardul Eroilor 7', N'Cluj-Napoca'),
 (N'Klausen Burger', N'Strada Regele Ferdinand 22', N'Cluj-Napoca'),
 (N'Grande Pizza', N'Strada Teodor Mihali 4', N'Cluj-Napoca'),
 (N'Chios Carrousel', N'Parcul Central', N'Cluj-Napoca'),
 (N'Via Restaurant', N'Strada Inocențiu Micu Klein 6', N'Cluj-Napoca'),
 (N'The Office Canteen', N'Bulevardul 21 Decembrie', N'Cluj-Napoca'),
 (N'Apahida Grill', N'Calea Dezmirului 1', N'Apahida');

3. **Insert Menu Items:** (50 entries - 2 per restaurant)

INSERT INTO MenuItems (RestaurantID, ItemName, Price, Category) VALUES

(1, N'Bean Soup', 28.00, N'Soup'),
 (1, N'Păpănași', 24.00, N'Dessert'),
 (2, N'Classic Burger', 42.00, N'Main'),
 (2, N'Cheesecake', 26.00, N'Dessert'),
 (3, N'Vegan Lasagna', 45.00, N'Vegan'),
 (3, N'Hummus Bowl', 34.00, N'Vegan'),
 (4, N'Pizza Diavola', 39.00, N'Pizza'),
 (4, N'Pizza Quattro Stagioni', 41.00, N'Pizza'),
 (5, N'Steak Sandwich', 55.00, N'Main'),
 (5, N'Duck Confit', 78.00, N'Main'),
 (6, N'Eggs Benedict', 32.00, N'Breakfast'),
 (6, N'Croissant', 12.00, N'Pastry'),

(7, N'Chicken Tikka', 48.00, N'Indian'),
(7, N'Garlic Naan', 12.00, N'Side'),
(8, N'Double Burger', 44.00, N'Main'),
(8, N'Onion Rings', 18.00, N'Side'),
(9, N'Garlic Pasta', 34.00, N'Main'),
(9, N'Bruschetta', 22.00, N'Starter'),
(10, N'Sushi Box', 65.00, N'Japanese'),
(10, N'Miso Soup', 18.00, N'Soup'),
(11, N'Crispy Box', 29.00, N'Fast Food'),
(11, N'Fries', 10.00, N'Side'),
(12, N'Margherita', 32.00, N'Pizza'),
(12, N'Salami Pizza', 37.00, N'Pizza'),
(13, N'Bacon Burger', 41.00, N'Main'),
(13, N'Sweet Potato', 19.00, N'Side'),
(14, N'Tochitură de Porc', 36.00, N'Romanian'),
(14, N'Mămăligă', 15.00, N'Side'),
(15, N'Noodle Box', 32.00, N'Asian'),
(15, N'Spring Rolls', 20.00, N'Asian'),
(16, N'Spicy Ramen', 38.00, N'Asian'),
(16, N'Green Tea', 14.00, N'Drink'),
(17, N'Avocado Toast', 30.00, N'Breakfast'),
(17, N'Omelette', 25.00, N'Breakfast'),
(18, N'Cappuccino', 15.00, N'Coffee'),
(18, N'Flat White', 16.00, N'Coffee'),
(19, N'Cold Brew', 18.00, N'Coffee'),
(19, N'Latte', 16.00, N'Coffee'),
(20, N'Klausen Burger', 45.00, N'Main'),
(20, N'Craft Beer', 22.00, N'Drink'),
(21, N'Prosciutto Pizza', 44.00, N'Pizza'),
(21, N'Calzone', 38.00, N'Pizza'),
(22, N'Grilled Duck', 62.00, N'Main'),
(22, N'Red Wine', 30.00, N'Drink'),
(23, N'Beef Tagliata', 85.00, N'Main'),
(23, N'Lava Cake', 28.00, N'Dessert'),
(24, N'Menu of the Day', 25.00, N'Lunch'),
(24, N'Lemonade', 15.00, N'Drink'),
(25, N'Mixed Grill', 48.00, N'Main'),
(25, N'Grilled Veggies', 18.00, N'Side');

4. Insert Orders: (25 entities)

INSERT INTO Orders (CustomerID, RestaurantID, DriverID, Status, OrderDate)
VALUES

(2, 1, 3, 'Delivered', '2026-01-04 14:14:41'),
(4, 2, 5, 'Delivered', '2026-01-04 14:14:41'),
(6, 3, 8, 'Pending', '2026-01-04 14:14:41'),
(7, 4, 11, 'Arriving', '2026-01-04 14:14:41'),
(9, 5, 14, 'Delivered', '2026-01-04 14:14:41'),
(10, 6, 17, 'Delivered', '2026-01-04 14:14:41'),
(12, 7, 20, 'Delivered', '2026-01-04 14:14:41'),
(13, 8, 23, 'Delivered', '2026-01-04 14:14:41'),
(15, 9, 3, 'Delivered', '2026-01-04 14:14:41'),
(16, 10, 5, 'Delivered', '2026-01-04 14:14:41'),
(18, 11, 8, 'Pending', '2026-01-04 14:14:41'),
(19, 12, 11, 'Arriving', '2026-01-04 14:14:41'),
(21, 13, 14, 'Pending', '2026-01-04 14:14:41'),
(22, 14, 17, 'Delivered', '2026-01-04 14:14:41'),
(24, 15, 20, 'Delivered', '2026-01-04 14:14:41'),
(2, 16, 23, 'Delivered', '2026-01-04 14:14:41'),
(4, 17, 3, 'Pending', '2026-01-04 14:14:41'),
(6, 18, 5, 'Pending', '2026-01-04 14:14:41'),
(7, 19, 8, 'Delivered', '2026-01-04 14:14:41'),
(9, 20, 11, 'Delivered', '2026-01-04 14:14:41'),
(10, 21, 14, 'Delivered', '2026-01-04 14:14:41'),
(12, 22, 17, 'Delivered', '2026-01-04 14:14:41'),
(13, 23, 20, 'Delivered', '2026-01-04 14:14:41'),
(15, 24, NULL, 'Pending', '2026-01-04 14:14:41'),
(16, 25, NULL, 'Pending', '2026-01-04 14:14:41');

5. Insert Order Details: (47 entries)

INSERT INTO OrderDetails (OrderID, ItemID, Quantity) VALUES

(1, 1, 2), (1, 2, 1),
(2, 3, 1), (2, 4, 1),
(3, 5, 1), (3, 6, 2),
(4, 7, 3), (4, 8, 1),

(5, 9, 1), (5, 10, 2),
(6, 11, 1), (6, 12, 2),
(7, 13, 1), (7, 14, 1),
(8, 15, 2), (8, 16, 1),
(9, 17, 1), (9, 18, 1),
(10, 19, 1), (10, 20, 1),
(11, 21, 2), (11, 22, 2),
(12, 23, 1), (12, 24, 1),
(13, 25, 1), (13, 26, 1),
(14, 27, 2), (14, 28, 1),
(15, 29, 1), (15, 30, 2),
(16, 31, 1), (16, 32, 1),
(17, 33, 1), (17, 34, 1),
(18, 35, 2),
(19, 37, 2),
(20, 39, 1), (20, 40, 2),
(21, 41, 1), (21, 42, 1),
(22, 43, 1), (22, 44, 2),
(23, 45, 1), (23, 46, 1),
(24, 47, 3),
(25, 49, 1), (25, 50, 2);

d. Queries:

I will now provide a list of queries that have real usage in the application built:

1. SELECT with JOIN:

GOAL: Display a list of all menu items and the restaurant they belong to.

```
SELECT r.Name AS Restaurant, m.ItemName, m.Price, m.Category
FROM Restaurants r
JOIN MenuItem m ON r.RestaurantID = m.RestaurantID
ORDER BY r.Name;
```

```

1  SELECT r.Name AS Restaurant, m.ItemName, m.Price, m.Category
2  FROM Restaurants r
3  JOIN MenuItems m ON r.RestaurantID = m.RestaurantID
4  ORDER BY r.Name;
5

```

214 % No issues found

Results Messages

	Restaurant	ItemName	Price	Category
1	Apahida Grill	Mixed Grill	48.00	Main
2	Apahida Grill	Grilled Veggies	18.00	Side
3	Bujole	Eggs Benedict	32.00	Breakfast
4	Bujole	Croissant	12.00	Pastry
5	Burger Island	Bacon Burger	41.00	Main
6	Burger Island	Sweet Potato	19.00	Side
7	Chios Carroussel	Grilled Duck	62.00	Main
8	Chios Carroussel	Red Wine	30.00	Drink
9	Cimbru Author Eatery	Steak Sandwich	55.00	Main
10	Cimbru Author Eatery	Duck Confit	78.00	Main
11	Eggcetera	Avocado Toast	30.00	Breakfast
12	Eggcetera	Omelette	25.00	Breakfast
13	Express Chef	Crispy Box	29.00	Fast Food
14	Express Chef	Fries	19.00	Side
15	Garlic Bites	Garlic Pasta	34.00	Main
16	Garlic Bites	Bruschetta	22.00	Starter
17	Grande Pizza	Prosciutto Pizza	44.00	Pizza
18	Grande Pizza	Calzone	38.00	Pizza
19	Indigo Indian	Chicken Tikka	48.00	Indian
20	Indigo Indian	Garlic Naan	12.00	Side
21	Klausen Burger	Klausen Burger	45.00	Main

Query executed successfully. DRAGOS-DELL (16.0 RTM) DRAGOS-DELL\drago (55) FoodDeliveryDB 00:00:00 50 rows

2. JOIN:

GOAL: Find all menu items for a specific restaurant.

```

SELECT ItemName, Price
FROM MenuItems m
JOIN Restaurants r ON m.RestaurantID = r.RestaurantID
WHERE r.Name = N'Zama';

```

```

1  SELECT ItemName, Price
2  FROM MenuItems m
3  JOIN Restaurants r ON m.RestaurantID = r.RestaurantID
4  WHERE r.Name = N'Zama';
5

```

214 % No issues found

Results Messages

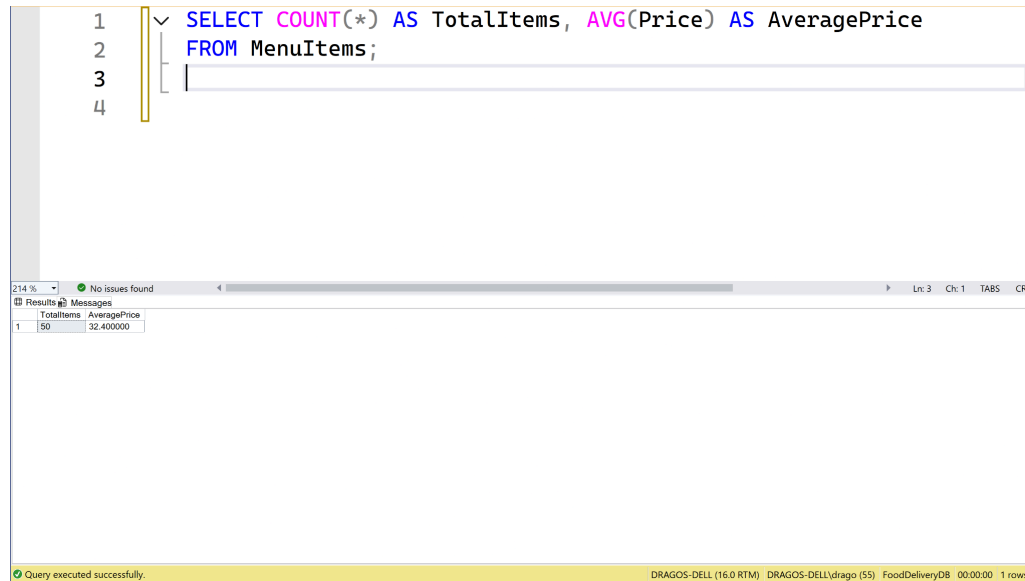
	ItemName	Price
1	Bean Soup	28.00
2	Păpănași	24.00

Query executed successfully. DRAGOS-DELL (16.0 RTM) DRAGOS-DELL\drago (55) FoodDeliveryDB 00:00:00 2 rows

3. AGGREGATE FUNCTIONS:

GOAL: Show the total number of items and the average price across the entire platform.

```
SELECT COUNT(*) AS TotalItems, AVG(Price) AS AveragePrice
FROM MenuItems;
```



The screenshot shows a SQL query editor with the following query:

```
1 SELECT COUNT(*) AS TotalItems, AVG(Price) AS AveragePrice
2 FROM MenuItems;
3
4
```

Below the query editor, the results are displayed in a table:

	TotalItems	AveragePrice
1	50	32.400000

At the bottom of the window, a status bar indicates: "Query executed successfully. DRAGOS-DELL (16.0 RTM) DRAGOS-DELL\drago (55) FoodDeliveryDB 00:00:00 1 rows".

4. GROUP BY with SUM:

GOAL: Calculate the total potential revenue each restaurant has generated from all orders.

```
SELECT r.Name, SUM(od.Quantity * mi.Price) AS TotalRevenue
FROM Restaurants r
JOIN Orders o ON r.RestaurantID = o.RestaurantID
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN MenuItems mi ON od.ItemID = mi.ItemID
GROUP BY r.Name
ORDER BY TotalRevenue DESC;
```

1	SELECT r.Name, SUM(od.Quantity * mi.Price) AS TotalRevenue
2	FROM Restaurants r
3	JOIN Orders o ON r.RestaurantID = o.RestaurantID
4	JOIN OrderDetails od ON o.OrderID = od.OrderID
5	JOIN MenuItems mi ON od.ItemID = mi.ItemID
6	GROUP BY r.Name
7	ORDER BY TotalRevenue DESC;
8	

Name	TotalRevenue
1 Zama	468.00
2 Cimbru Author Eatery	344.00
3 Bujoka	289.00
4 Tokyo Japanese	231.00
5 Wok'nRoll	228.00
6 Express Chef	184.00
7 Pizzeria Euphoria	158.00
8 Choco Carousel	122.00
9 Via Restaurant	113.00
10 Samara Foodhouse	113.00
11 MEAT UP	106.00
12 Klausen Burger	89.00
13 Traditional Apahida	87.00
14 Apahida Grill	84.00
15 Grande Pizza	82.00
16 The Office Canteen	75.00
17 Pizze Menia	69.00
18 Marty Restaurants	68.00
19 Indigo Indian	60.00
20 Burger Island	60.00
21 Garlic Bites	56.00

5. SELECT with SUBQUERY:

GOAL: Find menu items that are priced higher than the average price of all items.

SELECT ItemName, Price

FROM MenuItems

WHERE Price > (SELECT AVG(Price) FROM MenuItems);

1	SELECT ItemName, Price
2	FROM MenuItems
3	WHERE Price > (SELECT AVG(Price) FROM MenuItems);
4	
5	

ItemName	Price
2 Vegan Lasagna	45.00
3 Hummus Bowl	34.00
4 Pizza Diavola	39.00
5 Pizza Quattro Stagioni	41.00
6 Steak Sandwich	55.00
7 Duck Confit	78.00
8 Chicken Tikka	48.00
9 Double Burger	44.00
10 Garlic Pasta	34.00
11 Sushi Box	65.00
12 Salami Pizza	37.00
13 Bacon Burger	41.00
14 Tocitura de Porc	36.00
15 Spicy Ramen	38.00
16 Klausen Burger	45.00
17 Prosciutto Pizza	44.00
18 Calzone	38.00
19 Grilled Duck	62.00
20 Beef Tagliata	85.00
21 Mixed Grill	48.00

6. COMPLEX JOIN:

GOAL: Show the order history for a specific user, including restaurant and status.

SELECT o.OrderID, r.Name AS Restaurant, o.Status, o.OrderDate


```
FROM Orders o
JOIN Users u ON o.CustomerID = u.UserID
JOIN Restaurants r ON o.RestaurantID = r.RestaurantID
WHERE u.Username = 'maria_g';
```

The screenshot shows a SQL query editor with a query that filters orders for a user named 'maria_g'. Below the editor, a results window displays a table with 5 rows of order data. At the bottom, a status bar indicates the query was executed successfully.

```
1 SELECT o.OrderID, r.Name AS Restaurant, o.Status, o.OrderDate
2 FROM Orders o
3 JOIN Users u ON o.CustomerID = u.UserID
4 JOIN Restaurants r ON o.RestaurantID = r.RestaurantID
5 WHERE u.Username = 'maria_g';
6
```

OrderID	Restaurant	Status	OrderDate
3	Zama	Delivered	2026-01-04 14:16:41.473
16	Noodle Pack	Delivered	2026-01-04 14:16:41.473
1032	Tokyo Japanese	Delivered	2026-01-06 22:10:25.040
1033	Biggle	Pending	2026-01-07 00:45:54.353
1034	Combin Author Eatery	Pending	2026-01-07 00:50:51.157

Query executed successfully. DRAGOS-DELL (16.0 RTM) DRAGOS-DELL (16.0 RTM) FoodDeliveryDB 00:00:00 5 rows

7. AGGREGATE FUNCTIONS:

GOAL: Identify the most expensive and least expensive items in the "Pizza" category.

```
SELECT MAX(Price), MIN(Price)
FROM MenuItem
WHERE Category = N'Pizza';
```

```
1 SELECT MAX(Price), MIN(Price)
2 FROM MenuItems
3 WHERE Category = N'Pizza';
4
5
```

	(No column name)	(No column name)
1	44.00	32.00

Query executed successfully. DRAGOS-DELL (16.0 RTM) DRAGOS-DELL\drago (SS) FoodDeliveryDB 00:00:00 1 row

8. UPDATE:

GOAL: Assign a driver to a pending order and update its status.

UPDATE Orders

SET DriverID = 3, Status = 'Arriving'

WHERE OrderID = 24 AND Status = 'Pending';\

```
1 UPDATE Orders
2 SET DriverID = 3, Status = 'Arriving'
3 WHERE OrderID = 24 AND Status = 'Pending';
4
5
6
```

(1 row affected)

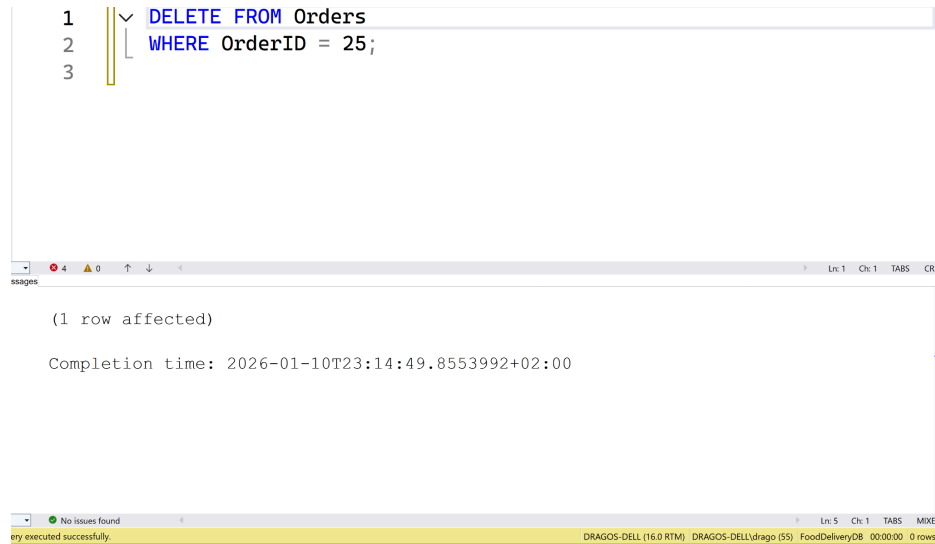
Completion time: 2026-01-10T23:12:10.3554673+02:00

Query executed successfully. DRAGOS-DELL (16.0 RTM) DRAGOS-DELL\drago (SS) FoodDeliveryDB 00:00:00 0 rows

9. DELETE with CASCADE CHECK:

GOAL: Remove a specific order from the system, removing as well all its details from OrderDetails.

DELETE FROM Orders
WHERE OrderID = 25;



The screenshot shows a SQL query execution window. The query is: `DELETE FROM Orders WHERE OrderID = 25;`. The execution results show "(1 row affected)" and a completion time of "2026-01-10T23:14:49.8553992+02:00". The status bar at the bottom indicates "Query executed successfully." and "0 rows" were returned.

```
1  DELETE FROM Orders
2  WHERE OrderID = 25;
3
```

(1 row affected)

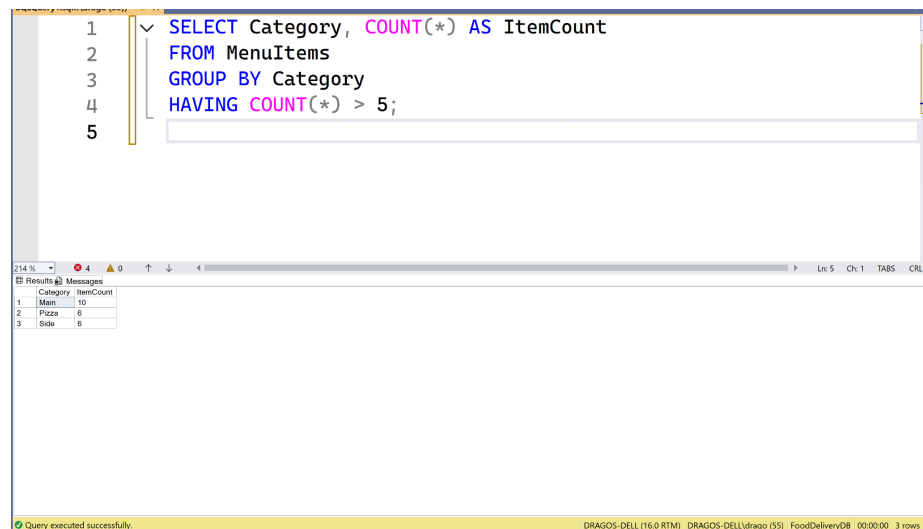
Completion time: 2026-01-10T23:14:49.8553992+02:00

Query executed successfully.

10. GROUP BY with HAVING:

GOAL: List categories that have more than 5 items listed across all restaurants.

SELECT Category, COUNT(*) AS ItemCount
FROM MenuItems
GROUP BY Category
HAVING COUNT(*) > 5;



The screenshot shows a SQL query execution window. The query is: `SELECT Category, COUNT(*) AS ItemCount FROM MenuItems GROUP BY Category HAVING COUNT(*) > 5;`. The execution results show a table with 3 rows: Main (10), Pizza (6), and Side (6). The status bar at the bottom indicates "Query executed successfully." and "3 rows" were returned.

```
1  SELECT Category, COUNT(*) AS ItemCount
2  FROM MenuItems
3  GROUP BY Category
4  HAVING COUNT(*) > 5;
5
```

Category	ItemCount
Main	10
Pizza	6
Side	6

Query executed successfully.

7. Conclusion

The FoodDeliveryDB project successfully establishes a normalized and scalable relational structure that accurately models the restaurant market in Cluj-Napoca. By implementing strict referential integrity and custom check constraints, the system ensures data consistency and protects against the entry of invalid records. This database provides a solid and efficient foundation for the corresponding Java application, demonstrating professional standards in data modeling and SQL implementation.